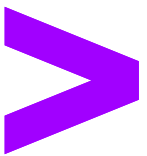


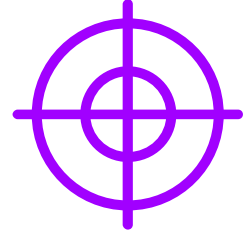
WebDriver API



Module Objectives

At the end of this course, you will be able to:

- Understand about WebDriver API
- Understand about Selenium Locators
- Understand about different WebDriver Commands
- Understand about different Web Element Commands
- Demonstrate how to handle drop-down menu
- Demonstrate how to use wait commands



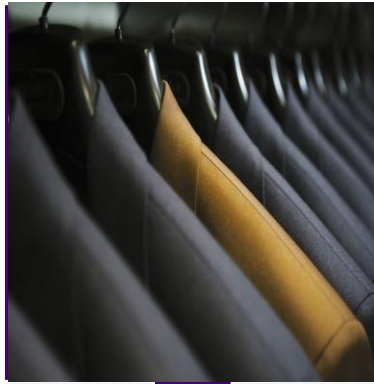
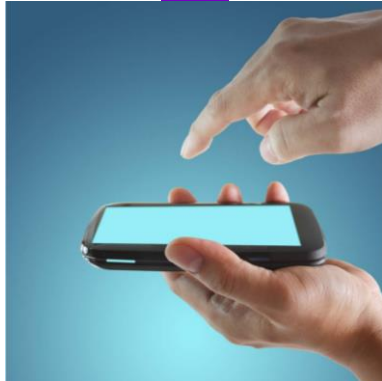
Ground Rules

For a successful class, please:



Arrive on time

Keep your cell phones off



Wear business formals

Assist and show respect to your colleagues irrespective of skill and knowledge level



Do not use the web for personal reasons during training sessions

Adhere to attendance policy as directed by training coordinator



WebDriver API

- 01** Overview of WebDriver API
- 02** Selenium Locators
- 03** WebDriver Commands
- 04** WebElement Commands
- 05** Select Commands
- 06** Wait Commands

Overview of WebDriver API

Overview of WebDriver API (1)

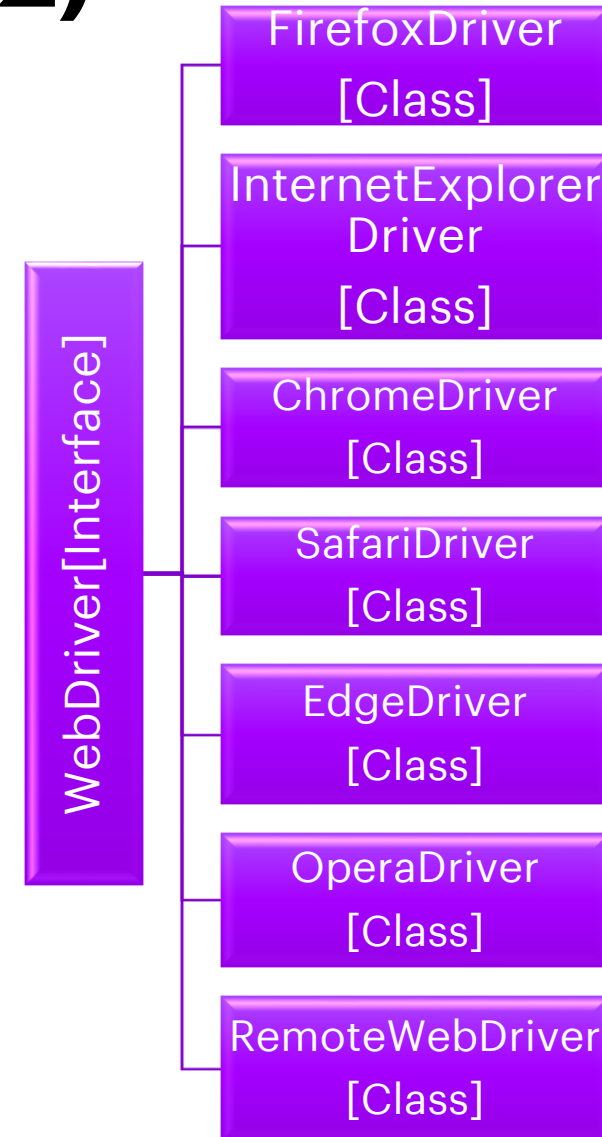
Introduction to WebDriver API

- Selenium WebDriver Library provides support for interactive web pages, where objects on a web page may change without reloading the page (example: AJAX-based web applications)
- WebDriver Library is an object-oriented API and provides support to cross-browser testing
- It makes direct calls to the browser for automation

Overview of WebDriver API (2)

Introduction to WebDriver API

- WebDriver is the main interface used for testing in Selenium, which represents an idealized web browser
- The methods in this interface fall into three categories:
 - Control of the browser
 - Selection of web elements
 - Assists in debugging

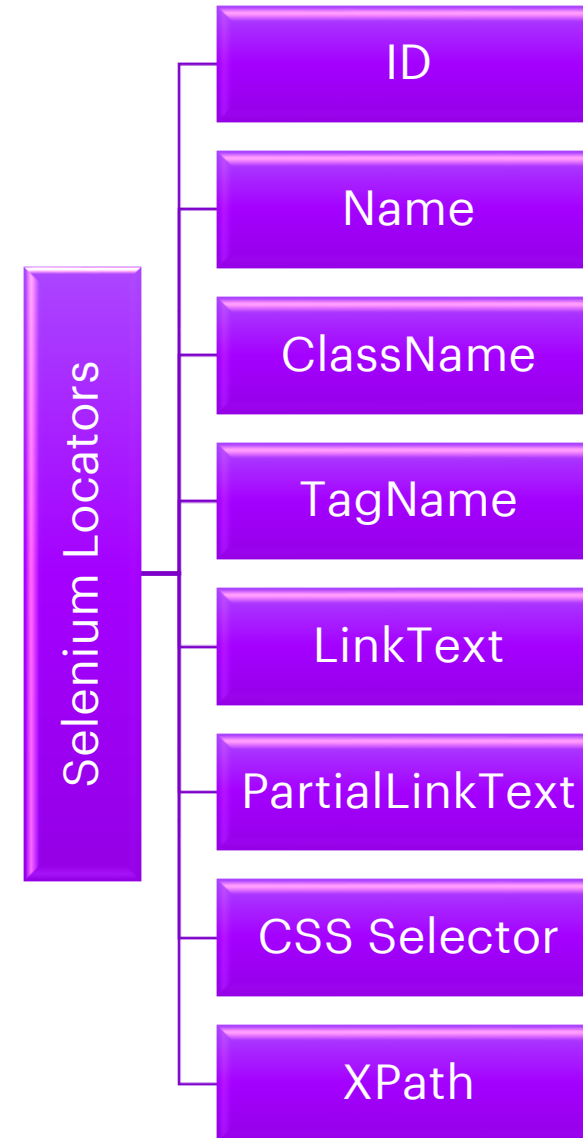


Selenium Locators

Selenium Locators (1)

Locating Web Elements using Selenium Locators

- Selenium WebDriver API provides a technique for locating elements on web pages



Selenium Locators (2)

Locating Web Element By ID

- ID locator is used to identifying the element using the ID attribute
- It can be used only if a web element contains an ID attribute

```
<input type="text" id="userName"/>
```

Consider this element
which has an ID attribute

- Syntax to locate this element using ID locator

```
WebElement unameTextBox = driver.findElement(By.id("userName"));
```

Pros: The ID attribute of an element is always unique on a webpage. It is a faster and most reliable way to locate the objects from DOM

Cons: The ID locator is only suitable for elements with static IDs. If the element has a dynamic ID attribute, the ID locator will not be useful

Selenium Locators (3)

Locating Web Element By Name

- Name locator is used to identifying the element using the Name attribute
- It can be used only if a web element contains a Name attribute

Consider this element
which has a Name attribute

```
<input type="text" name="password"/>
```

- Syntax to locate this element using Name locator

```
WebElement pwdTextBox = driver.findElement(By.name("password"));
```

Note: If multiple elements have the same name, Selenium WebDriver will locate the first occurrence of the element

Selenium Locators (4)

Locating Web Element By ClassName

- ClassName locator is used to identify the element using the Class attribute
- It can be used only if a web element contains a Class attribute

Consider this element
which has a Class attribute

```
<input type="text" class="xyz_gi"/>
```

- Syntax to locate this element using ClassName locator

```
WebElement element = driver.findElement(By.className("xyz_gi"));
```

Note: When the class name has spaces (for example class="xyz gi"), then we must take the first word of the class name instead of the full name. Otherwise, it displays an error message

Selenium Locators (5)

Locating Web Element By TagName

- This strategy is used to locate the elements using the tag name
- List<WebElement> can be used to store multiple elements

Example 1:

- To find the number of rows displayed in a web table, by locating all <tr> tags

```
List<WebElement> allRows = driver.findElements(By.tagName("tr"));  
int noOfRows = allRows.size();
```

Example 2:

- To find the number of rows displayed in a web table, by locating all <tr> tags

```
List<WebElement> allImages = driver.findElements(By.tagName("img"));  
int noOfImages = allImages.size();
```

Selenium Locators (6)

Locating Web Element By LinkText

- LinkText locator is used to identify the anchor element with text (Hyperlinks)
- The text associated with hyperlinks is used in linktext locator

```
<a href="/demo/login.php">User Login</a>
```

Consider this element which is inside anchor tag <a>

- Syntax to locate this element using LinkText locator

```
WebElement loginLink = driver.findElement(By.LinkText("User Login"));
```

Pros: This locator will select only anchor elements, which can be used for testing page navigation

Cons: It is a must to know the link text before the links are identified. Not suitable for links with dynamic text

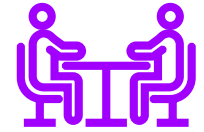
Selenium Locators (7)

Locating Web Element By PartialLinkText

- This strategy is used to locate the anchor element with dynamic text (Hyperlinks)
- The partial text associated with hyperlinks is used in partial link text locator
- For example, there is a link **"Inbox(125)"**. This link contains the text (number of new emails, which may change dynamically). To locate this element, a partial link text locator can be used by specifying the fixed portion of the link text
- Syntax to locate this element using PartialLinkText locator

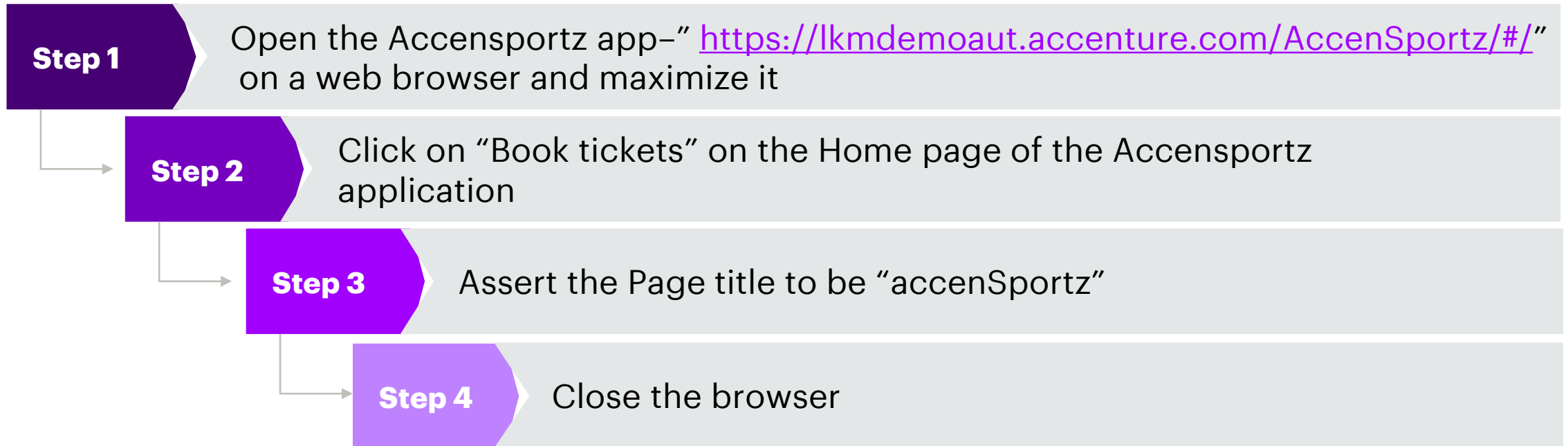
```
WebElement inboxLink = driver.findElement(By.partialLinkText("Inbox"));
```


Selenium Locators (8)



Demo: Automate a scenario with Selenium Locators

- Samantha wants to automate a scenario utilizing selenium Locators.
- Scenario



Selenium Locators (9)



Demo Solution: Automate a scenario with Selenium Locators

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class SeleniumLocators {

    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.quit();
    }
}
```

Selenium Locators (10)



Demo Solution: Automate a scenario with Selenium Locators

```
@Test
Run | Debug
public void f() {
    // clicking on book tickets
    WebElement booktickets = driver.findElement(By.linkText("Book Tickets"));
    booktickets.click();

    // Asserting the Title
    Assert.assertEquals(driver.getTitle(), "accenSportz");
}
```

Selenium Locators (11)



Demo Result: Automate a scenario with Selenium Locators

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 28, 2024 4:51:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 130, returning the closest version;
PASSED: test.SeleniumLocators.f
```

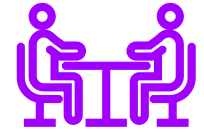
```
=====
Default test
```

```
Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
```

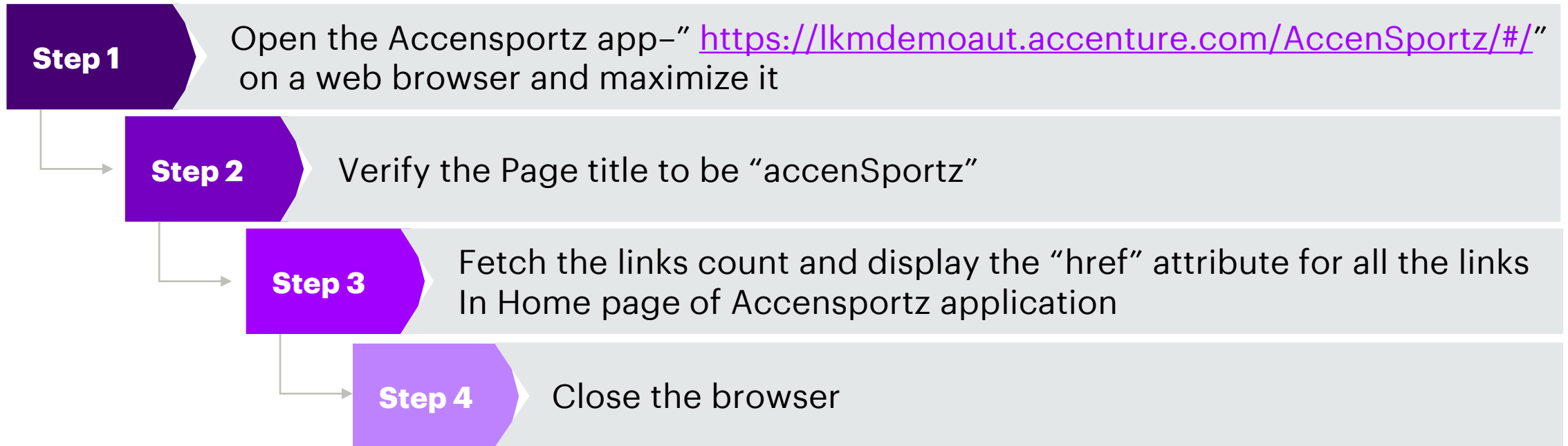
```
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

Selenium Locators (12)



Demo: Automate a scenario with Selenium Locators

- Samantha wants to automate a scenario utilizing selenium Locators.
- Scenario



Selenium Locators (13)



Demo Solution: Automate a scenario with Selenium Locators

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class SeleniumLocators {

    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.quit();
    }
}
```

Selenium Locators (14)



Demo Solution: Automate a scenario with Selenium Locators

```
@Test
public void f() {

    // Asserting the Title
    Assert.assertEquals(driver.getTitle(), "accenSportz");

    // Locating links
    List<WebElement> alllinks = driver.findElements(By.tagName("a"));
    int linkscout=alllinks.size();
    System.out.println("total count: " + linkscout);

    // retrieving all links through for loop
    for (int i = 0; i < linkscout; i++) {
        String textvalue = alllinks.get(i).getText();
        String hrefvalue = alllinks.get(i).getAttribute("href");
        System.out.println(textvalue + ": " + hrefvalue);
    }
}
```

Selenium Locators (15)



Demo Result: Automate a scenario with Selenium Locators

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 28, 2024 6:49:50 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 130, returning the closest version; found: 126;
total count: 8
: javascript:void(0)
AccenSportz: https://lkmdemoaut.accenture.com/AccenSportz/#/
Player Registration: https://lkmdemoaut.accenture.com/AccenSportz/#/webelements/form
Player & Team Strategy: https://lkmdemoaut.accenture.com/AccenSportz/#/actions/playerandteamstrategy
Book Tickets: https://lkmdemoaut.accenture.com/AccenSportz/#/webelements/tickets
Sports Coaching: https://lkmdemoaut.accenture.com/AccenSportz/#/widgets/sportscoaching
Fantasy Sports: https://lkmdemoaut.accenture.com/AccenSportz/#/widgets/fantasysports
Sports Merchandise: https://lkmdemoaut.accenture.com/AccenSportz/#/frames/merchandise
PASSED: test.Linkscount.f
```

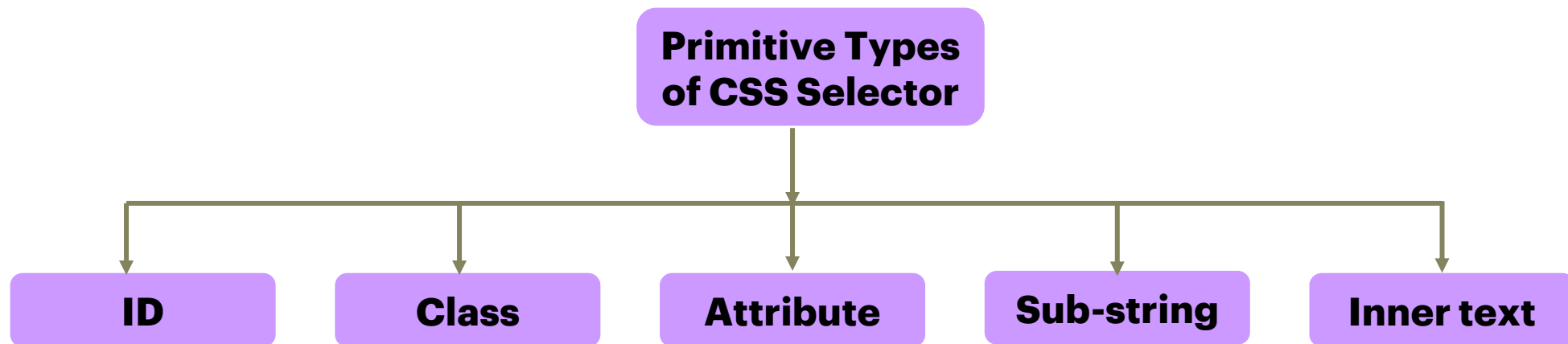
```
=====
Default test
Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```


Selenium Locators (16)

Locating Web Element By CSS Selector

- This strategy is used to locate the element by creating its CSS selector
- It is faster, more readable, and used more often



Selenium Locators (17)

CSS Selector with ID

- This strategy is used to locate the element using the CSS selector type with an ID attribute

```
<input type="text" id="username"/>
```

Consider this web element
which has an ID attribute

- Syntax to locate this element using CSS Selector with ID attribute

```
WebElement unameElement = driver.findElement(By.cssSelector("input#username"));
```

Selenium Locators (18)

CSS Selector with Class

- This strategy is used to locate the element using the CSS selector type with a Class attribute
- We use this locator type when the element has only class attribute

```
<input type="text" class="xyz"/>
```

Consider this web element
which only has Class attribute

- Syntax to locate this element using CSS Selector with Class attribute

```
WebElement unameElement = driver.findElement(By.cssSelector("input.xyz"));
```

Selenium Locators (19)

CSS Selector with attribute

- This strategy is used to locate the element using the CSS selector type with any attribute.
- We use this locator type when the element have an attribute like type, value etc.

```
<input type="submit"/>
```

Consider this web element
which only has type attribute

- Syntax to locate this element using CSS Selector with any other attribute

```
WebElement submitBtn = driver.findElement(By.cssSelector("input[type='submit']"));
```

Selenium Locators (20)

CSS Selector with multiple attribute

- Use a CSS selector with multiple attribute locator type when
 - the element does not have a unique attribute
 - the single attribute is not sufficient to identify the element

Consider this web element

```
<div class="ajax_enabled" style="display:block">
```

- Syntax to locate this element using CSS Selector with multiple attributes

```
WebElement submitBtn = driver.findElement(By.cssSelector("div[class='ajax_enabled'][style='display:block']"));
```

Selenium Locators (21)

CSS Selector with Sub-String

- This strategy allows matching of partial string and is very useful with attributes having dynamic values
- Supports mechanism like starts-with, ends-with, and contains

- Match a prefix (starts-with)

`<div id="empId_456">` → `css="div[id^='empId']"`

- Match a prefix (ends-with)

`<div id="123_empId">` → `css="div[id$='empId']"`

- Match a sub-string (contains)

`<div id="123_empId_456">` → `css="div[id*='_empId_']"`
`css="div:contains(_empId_)"`

Selenium Locators (22)

CSS Selector for Child Elements

- Consider a list element which has a list of child elements like below

```
<ul id="fruit">  
  <li>Apple</li>  
  <li>Orange</li>  
  <li>Banana</li>  
</ul>
```

- To locate the first child element: we can use **first-child** or **first-of-type**

```
css="ul#fruit li:first-of-type"
```

- To locate 2nd child element: We can use **nth-child()** or **nth-of-type()**

```
css="ul#fruit li:nth-of-type(2)"
```

- To locate the last child-element: We can use **last-child** or **last-of-type**

```
css="ul#fruit li:last-child"
```

Selenium Locators (23)

Locating Web Element by XPath

- XPath is a query language for selecting nodes from an XML document. All the major browsers support XPath and HTML pages are represented as XHTML documents in DOM
- Selenium supports XPath for locating elements using XPath expressions or queries
- Locating elements with XPath works very well with a lot of flexibility. However, this is the least preferable locator strategy due to its slow performance
- One of the important differences between XPath and CSS is, with XPath we can search elements backward or forward in the DOM hierarchy while CSS works only in a forward direction. This means we can locate a parent element using a child element with XPath

Selenium Locators (24)

XPath Types

- There are two different types of XPath that can be used to locate an element on a web page

XPath Types

```
graph TD; A[XPath Types] --> B[Absolute XPath]; A --> C[Relative XPath];
```

Absolute XPath

- It provides direct access and complete path of an element
- It starts with single slash(/)
- Main disadvantage is, any change in the path of an element, makes the XPath failed
- Example:
`html/body/div[1]/section/div[1]/div/div/div/div[1]/div/div/div/div/div[3]/div[1]/div/h4[1]/b`

Relative XPath

- It starts from middle of DOM structure
- It starts with double slash (//) means anywhere on the web page
- Example: `//input[@name='uid']`

Selenium Locators (25)

Locating Web Element by XPath

Description	Xpath Example
Finding elements with absolute path	/html/body/form
Finding elements with relative path	//form
Finding elements using index	//form/input[1]
Finding elements using single attribute value	//input[@name='username']
Finding elements using multiple attributes values	• //input[@name='username'][@class='required'] • //input[@name='username' and @class='required'] • //input[@name='username' or @class='required']

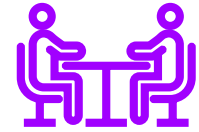
Selenium Locators (26)

Locating Web Element by XPath

- Performing a partial match on attribute values:

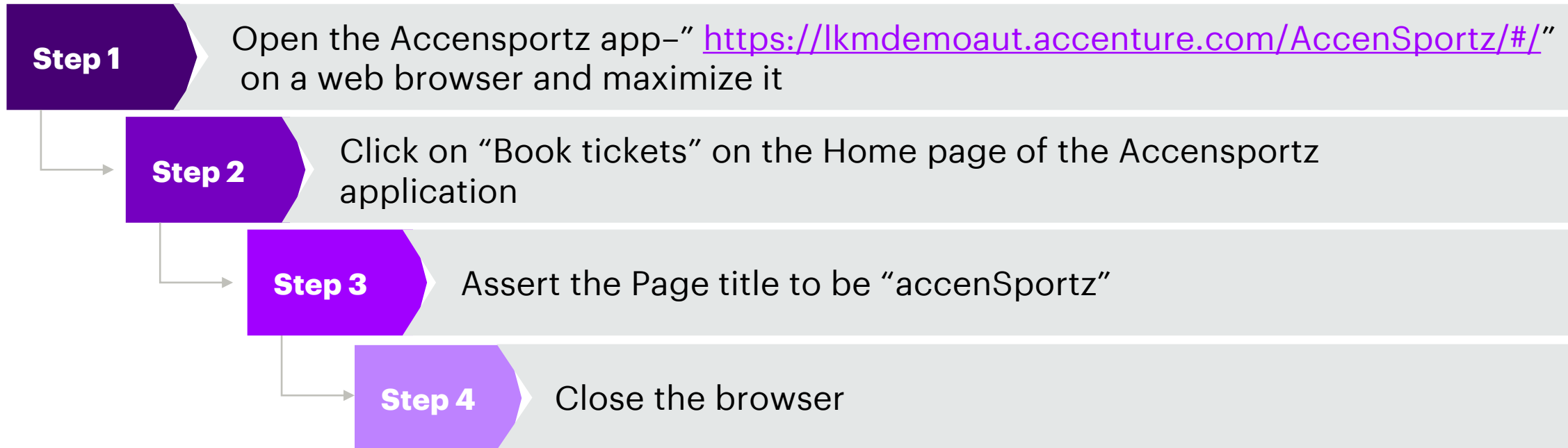
Syntax	Example	Description
starts-with()	XPath=//input[starts-with(@id,'ctrl')]	Starting with: For example, if the ID of an element is ctrl_12,this will locate and return elements with ctrl at the beginning of the ID.
ends-with()	XPath=//input[ends-with(@id,'_username')]	Ending with: For example, if the ID of an element is a1_username, this will locate and return elements with_username at the end of the ID
contains()	XPath=//input[contains(@id,'username')]	For example, if the ID for an element is panel_login_userName_textfield, this will use the username part in the middle to match and locate the element

Selenium Locators (27)



Demo: Automate a scenario with Selenium Locators- XPath

- Samantha wants to automate a scenario utilizing XPath selenium Locators.
- Scenario



Selenium Locators (28)



Demo Solution: Automate a scenario with Selenium Locators

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class SeleniumLocators {

    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.quit();
    }
}
```

Selenium Locators (29)



Demo Solution: Automate a scenario with Selenium Locators

```
@Test
Run | Debug
public void f() {
    // clicking on book tickets
    WebElement booktickets = driver.findElement(By.xpath("//a[@name='booktickets']"));
    booktickets.click();

    // Asserting the Title
    Assert.assertEquals(driver.getTitle(), "accenSportz");
}
```

Selenium Locators (30)



Demo Result: Automate a scenario with Selenium Locators

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 28, 2024 4:51:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 130, returning the closest version;
PASSED: test.SeleniumLocators.f
```

```
=====
Default test
```

```
Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
```

```
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

Relative Locators (1)

Relative Locators

- Selenium 4 has introduced a new feature called “Relative Locators”
- This feature allows us to identify and locate elements that are close to each other, based on their visual location in DOM
- On the technical side, Selenium uses the `getBoundingClientRect()` function to find the relative elements by using properties like left, right, top, and bottom
- Relative locator can be used by implementing `findElement()` method with “`withTagName()`” method
- Note:
 - These locators might not work with overlapped elements
 - Use selenium 4 for stable behavior of relative locators
- Learn more about Relative Locators from <https://www.selenium.dev/documentation/webdriver/elements/locators/>



Relative Locators (2)

Available Relative Locators

- They are very useful for unstable applications, elements not having unique attributes, and elements with dynamic values
- This table describes the following:

Relative Locator	Description
above()	Locate an element situated above the reference element
below()	Locate an element situated below the reference element
toLeftOf()	Locate an element situated to the left of the reference element
toRightOf()	Locate an element situated to the right of the reference element
near()	Locate an element situated 50 pixels away from the reference element



Relative Locators (3)

Syntax of Relative Locators

```
// Locate the reference element "Please fill your feedback here:"
WebElement referenceLabel =
    driver.findElement(By.id("fill_form"));

// Locate "First Name" just below the referenceLabel
WebElement firstNameElement =
    driver.findElement(withTagName("input")
        .below(referenceLabel));

// Locate "Last Name" right of firstNameElement just below the referenceLabel
WebElement lastNameElement =
    driver.findElement(withTagName("input")
        .toRightOf(firstNameElement)
        .below(referenceLabel));

// Locate "Mobile No" just below the lastNameElement
WebElement mobileNoElement =
    driver.findElement(withTagName("input")
        .below(lastNameElement));

// Locate "Email Address" left of mobileNoElement just below the firstNameElement
WebElement emailElement =
    driver.findElement(withTagName("input")
        .toLeftOf(mobileNoElement)
        .below(firstNameElement));
```

User Feedback Form

Please fill your feedback here:

Write your feedback here...

Send Feedback



Relative Locators (4)

Run All

```
11 public class RelativeLocatorsExample {
12     @Test
13     public void f() {
14
15         WebDriver driver = new EdgeDriver();
16
17         driver.get("https://www.selenium.dev/");
18         driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
19         driver.manage().window().maximize();
20
21         WebElement referenceElement = driver.findElement(By.xpath("//span[contains(text(), 'Downloads')]"));
22
23         WebElement aboveElement = driver.findElement(RelativeLocator.with(By.tagName("a")).above(referenceElement));
24         WebElement belowElement = driver.findElement(RelativeLocator.with(By.tagName("a")).below(referenceElement));
25         WebElement nearElement = driver.findElement(RelativeLocator.with(By.tagName("a")).near(referenceElement));
26         WebElement leftElement = driver.findElement(RelativeLocator.with(By.tagName("a")).toLeftOf(referenceElement));
27         WebElement rightElement = driver.findElement(RelativeLocator.with(By.tagName("a")).toRightOf(referenceElement));
28
29         System.out.println("Above: " + aboveElement.getText());
30         System.out.println("Below: " + belowElement.getText());
31         System.out.println("Near: " + nearElement.getText());
32         System.out.println("Left: " + leftElement.getText());
33         System.out.println("Right: " + rightElement.getText());
34     }
35 }
```



WebDriver Commands

WebDriver Commands (1)

WebDriver Commands

There is a many of Selenium WebDriver commands which will help us to **perform the action** on a web browser

Get Methods → Retrieve information about the current page

- **driver.get("Application URL")** sends the AUT URL to the browser
- **driver.getTitle()** gets the title of the current page
- **driver.getCurrentUrl()** gets the current URL of the page which is loaded on the browser
- **driver.getPageSource()** gets the source code of the web page

Find WebElement Methods → Retrieve the desired element by specifying the locator

- **driver.findElement()** command finds the first Web Element
- **driver.findElements()** command finds all elements within the current web page

WebDriver Commands (2)

Navigate Commands

Selenium WebDriver Navigation commands which will help us to **perform the action** on a web browser

Navigate Methods → Used to navigate WebDriver object from one page to another

- **navigate().to()** method is like driver.get() command. To("Application URL") sends the URL to the browser
- **navigate().back()** will go back to the previous page on the browser's history
- **navigate().forward()** method takes the driver forward by one page on the browser's history
- **navigate().refresh()** reloads the current page

WebDriver Commands (3)

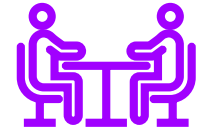
Close Commands

Selenium WebDriver close commands which help us to **close or quit** the web browser.

Close the Browser Methods

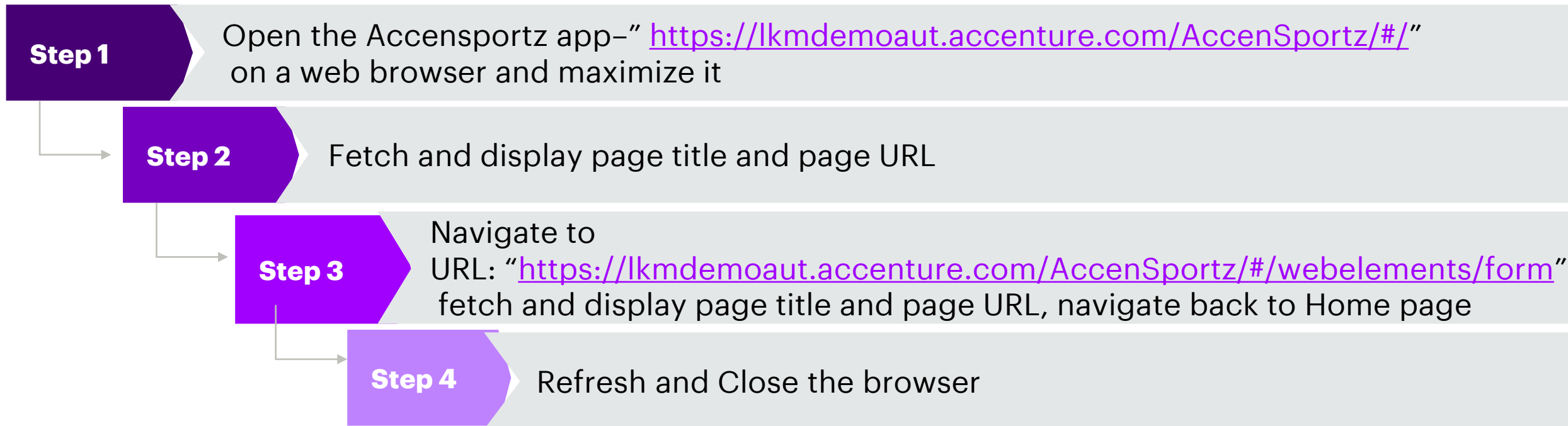
- **driver.close()** closes the current instance/window of the browser
- **driver.quit()** method is used to close all browser windows, opened by WebDriver for that instance

WebDriver Commands (4)



Demo: Automate a scenario using WebDriver Commands

- Samantha wants to automate a scenario utilizing WebDriver commands.
- Scenario



WebDriver Commands (5)



Demo Solution: Automate a scenario using WebDriver Commands

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class webdrivercommands {

    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.quit();
    }
}
```

WebDriver Commands (6)



Demo Solution: Automate a scenario using WebDriver Commands

```
@Test
Run | Debug
public void f() {
    System.out.println("page title: " + driver.getTitle());
    System.out.println("page URL: " + driver.getCurrentUrl());

    driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/webelements/form");
    System.out.println("page title: " + driver.getTitle());
    System.out.println("page URL: " + driver.getCurrentUrl());

    driver.navigate().back();
    driver.navigate().refresh();
}
```

WebDriver Commands (7)



Demo Result: Automate a scenario using WebDriver Commands

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 28, 2024 7:30:44 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 130, returning the closest version;
page title: accenSportz
page URL: https://lkmdemoaut.accenture.com/AccenSportz/#/
page title: accenSportz
page URL: https://lkmdemoaut.accenture.com/AccenSportz/#/webelements/form
PASSED: test.webdrivercommands.f
```

```
=====
Default test
```

```
Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
```

```
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

WebElement Commands (8)

Accessing the WebElement in a Form

- **SendKeys():** It is used to send text to the text field where the input type is text

```
driver.findElement(By.name("username")).sendKeys("selenium");
```

- **Clear():** It is used to clear the text.

```
driver.findElement(By.name("username")).clear();
```

- **getText():** It is used to get the text of an element

```
driver.findElement(By.id("orderId")).getText();
```

- **Click():** It is used to toggle the element on/off (Radio, Checkbox) and work with clickable elements (link, button)

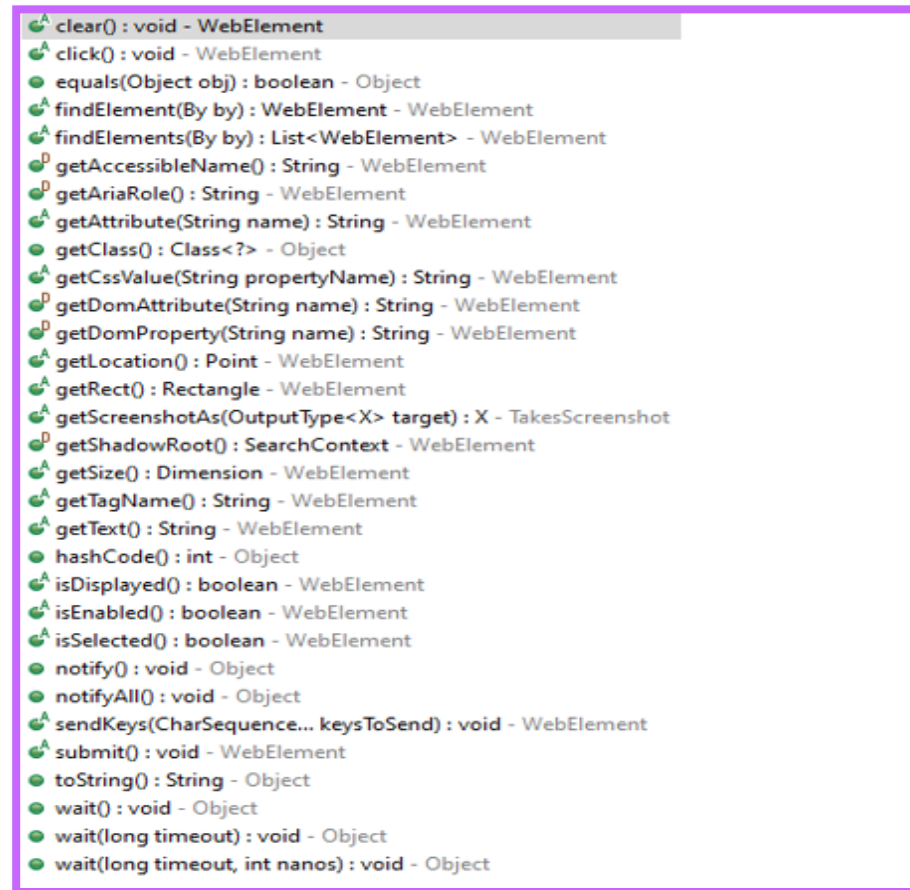
```
driver.findElement(By.cssSelector("input[value='Business']")).click();
```

```
driver.findElement(By.LinkText("Register here")).click();
```

WebElement Commands (9)

Other Commands:

- In Selenium Web Driver , there are several commonly used Web element commands and actions . The following screen shot displays the eclipse web element command panel.

A screenshot of the Eclipse IDE's API Explorer showing the Selenium WebElement interface. The list of methods is as follows:

```
clear() : void - WebElement
click() : void - WebElement
equals(Object obj) : boolean - Object
findElement(By by) : WebElement - WebElement
findElements(By by) : List<WebElement> - WebElement
getAccessibleName() : String - WebElement
getAriaRole() : String - WebElement
getAttribute(String name) : String - WebElement
getClass() : Class<?> - Object
getCssValue(String propertyName) : String - WebElement
getDomAttribute(String name) : String - WebElement
getDomProperty(String name) : String - WebElement
getLocation() : Point - WebElement
getRect() : Rectangle - WebElement
getScreenshotAs(OutputType<X> target) : X - TakesScreenshot
getShadowRoot() : SearchContext - WebElement
getSize() : Dimension - WebElement
getTagName() : String - WebElement
getText() : String - WebElement
hashCode() : int - Object
isDisplayed() : boolean - WebElement
isEnabled() : boolean - WebElement
isSelected() : boolean - WebElement
notify() : void - Object
notifyAll() : void - Object
sendKeys(CharSequence... keysToSend) : void - WebElement
submit() : void - WebElement
toString() : String - Object
wait() : void - Object
wait(long timeout) : void - Object
wait(long timeout, int nanos) : void - Object
```

Select Commands

Select Commands (1)

Handle Drop Down Menu

- Selenium WebDriver provides a built-in class called **Select** which is used to handle drop-down menu on a web page
- To handle drop-down menu:
 - Import the package org.openqa.selenium.support.ui.Select
 - Instantiate the drop-down menu as a “**Select**” object in WebDriver.
- **Syntax** : Select obj_name_Select = new Select();

```
<select id="products-orderby" name="products-orderby" onchange="setLocation(this.value);">  
  <option selected="selected" value="http://demowebshop.tricentis.com/desktops?orderby=0">Position</option>  
  <option value="http://demowebshop.tricentis.com/desktops?orderby=5">Name: A to Z</option>  
  <option value="http://demowebshop.tricentis.com/desktops?orderby=6">Name: Z to A</option>  
  <option value="http://demowebshop.tricentis.com/desktops?orderby=10">Price: Low to High</option>  
  <option value="http://demowebshop.tricentis.com/desktops?orderby=11">Price: High to Low</option>  
  <option value="http://demowebshop.tricentis.com/desktops?orderby=15">Created on</option>  
</select>
```

Sort By
Position
drop-down
menu

Syntax to select a value from Sort By Position drop-down menu

```
Select Sort_Position = new Select (driver.findElement(By.id("products-orderby")));  
// use any of method to select the sort by position drop down  
Sort_Position.selectByValue("Price: Low to High");  
Sort_Position.selectByVisibleText("Created on");  
Sort_Position.selectByIndex(2);
```

Select Commands (2)

Select Class Methods

`selectByVisibleText()` and `deselectByVisibleText()` : Selects/deselects the option matching the text

`selectByValue()` and `deselectByValue()` :Selects/deselects the option matching the value

`selectByIndex()` and `deselectByIndex()` : Selects/deselects the option matching the index

`isMultiple()` : Returns TRUE / FALSE to check multiple selection on option

`deselectAll()` :All the selected entries are cleared.

Wait Commands

Wait Commands (1)

Need of Wait Commands

- Selenium WebDriver is designed to search for desired elements on the web page
- WebDriver searches the element by its locator for a default period. If the search is unsuccessful, WebDriver gives a **NoSuchElementException**
- Sometimes, the web page loads slowly due to network issues or performance issues of the webserver. In such cases, WebDriver is not able to search for the elements, since the web page is not loaded completely in time
- In such a situation, WebDriver must be provided with some extra time to search for the elements. We must design the Selenium WebDriver script to synchronize the tests
- There are 2 types of Wait commands:
 - Implicit Wait
 - Explicit Wait

Wait Commands (2)

Implicit Wait

- An implicit wait polls DOM for a certain amount of time when trying to find an element or elements if they are not immediately available
- However, an implicit wait may slow down your tests when an application responds normally, as it will wait for each element appearing in DOM and increases overall execution time

Example:

```
WebDriver driver = new FirefoxDriver();  
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);  
  
driver.get("http://newtours.demoaut.com/");  
WebElement myDynamicElement = driver.findElement(By.id("myDynamicElement"));
```

Wait Commands (3)

Explicit Wait

- Explicit Wait is used for synchronizing tests on a particular web element, which provides better control when compared with an implicit wait
- It is the code defined to wait for a certain condition to occur before proceeding to further execution of test steps
- It can only be implemented in cases where synchronization is required only for a specific WebElement
- The Selenium WebDriver provides library classes like **WebDriverWait** and **ExpectedConditions** for implementing an explicit wait.
- Used mainly for AJAX-based web elements

```
WebDriverWait wait=new WebDriverWait(driver,Duration.ofSeconds(20));  
wait.until(ExpectedConditions.presenceOfElementLocated(By.partialLinkText("SignIn")));
```

- The above code waits up to 20 seconds before throwing a Timeout Exception or if the expected condition is met, it will return the element in 0 - 20 seconds

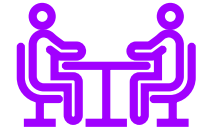
Wait Commands (4)

Explicit Wait

- The **ExpectedConditions** class provides a set of predefined conditions to wait before proceeding further in the code
- Some common conditions frequently used in automating web browsers supported by **ExpectedConditions** class:

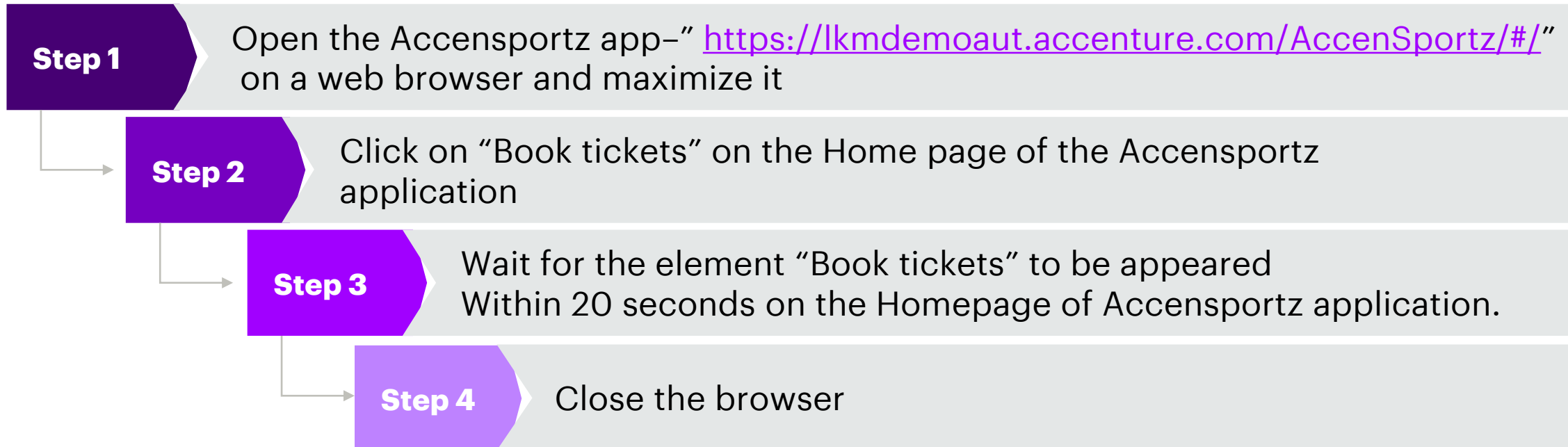
Pre-defined Condition	Selenium Method
An element is visible and enabled	elementToBeClickable(By Locator)
An element is selected	elementToBeSelected(WebElement element)
Presence of an element	presenceOfElementLocated(By Locator)
Specific text present in an element	textToBePresentInElement(By Locator, String text)
Element Value	textToBePresentInElementValue(By Locator, String text)
Title	titleContains(string title)

Wait Commands (5)



Demo: Automate a scenario with wait commands

- Samantha wants to automate a scenario utilizing wait commands.
- Scenario



Wait Commands (6)



Demo Solution: Automate a scenario with Wait commands

```
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

Run All
public class waitcommands {
    WebDriver driver;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        driver.quit();
    }
}
```

Wait Commands (7)



Demo Solution: Automate a scenario with Wait commands

```
@Test
Run | Debug
public void f() {

    // wait for the element
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    wait.until(ExpectedConditions.presenceOfAllElementsLocatedBy(By.linkText("Book Tickets")));

    // clicking on book tickets
    driver.findElement(By.xpath("//a[@name='booktickets']")).click();

}
```


Wait Commands (8)



Demo Solution: Automate a scenario with Wait commands

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 28, 2024 8:01:35 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 130, returning the closest version
PASSED: test.waitcommands.f
```

```
=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

Knowledge Check

Question 01



Which one of the following makes direct calls to browser for automation?

a) Selenium RC

b) Selenium WebDriver

Knowledge Check

Question 02



For the element, **Click Me**. Which is the correct locator(s) to use?

a) By.linkText("Click Me")

a) By.linkText("Click")

b) By.partialLinkText("Cme")

a) By.partialLinkText("Click-Me")

Knowledge Check

Question 03



Which of the following Wait is used for a particular Element?

a) Explicit Wait

b) Implicit Wait

Module Summary

Now, you will be able to:

- Understand about WebDriver API
- Understand about Selenium Locators
- Understand about different WebDriver Commands
- Understand about different Web Element Commands
- Demonstrate how to handle drop-down menu
- Demonstrate how to use wait commands



Thank You

